

HeatMAP: A Full-System Study of Sub-Seasonal Heatwave Forecasting for Thailand

Machine Learning, Data Contract, and Serverless Deployment

MCTEEKUNG

github.com/MCTEEKUNG/DeepSeek_Heatwave · github.com/MCTEEKUNG/heatwave-contract
github.com/MCTEEKUNG/HeatMAP-Frontend · github.com/MCTEEKUNG/Heatwave_Backend_Elysia

Preprint — June 20, 2026 · *Revised June 20, 2026 (corrected revision)*

Revision note. This revision corrects discrepancies found by an audit against the `DeepSeek_Heatwave` source repository, with no change to scientific content or conclusions: (1) the embargo gap is stated consistently as **49 days** (6-week maximum lead = 42 days + 7-day target window) throughout — earlier headline text said "42-day embargo", contradicting the Methods and the code (`train.py GAP=49`); (2) §3.2 now states that the **deployed** per-province model is `logistic_balanced_cal` (balanced weighting + Platt calibration), consistent with Table 4 and the published contract, while the unweighted variant is the registry default; (3) the feature-table size is expressed in daily forecast-origination dates ($\approx 840,000$ rows). Numbers for the alternative `Heatwave_Backend_Elysia` deployment (§10), the readiness audit (Table 9), and the map (Fig. 11) are preserved as in the original and were not independently re-verified in this revision.

Abstract. HeatMAP is a deployed end-to-end system that issues weekly probabilistic heatwave forecasts for all 77 provinces of Thailand at lead times of 2–4 weeks. The system spans four git repositories: a Python ML pipeline that ingests ERA5 reanalysis, MJO RMM and Niño 3.4 indices and trains per-province Logistic Regression and LightGBM models with Platt calibration; a static JSON data contract that decouples the model from a TypeScript Expo web frontend; and a publish-bridge that gates every release behind a hard validator plus a five-category production-readiness gate. This paper documents the full system as a case study in "responsible ML for rare-event forecasting". We make four contributions. (1) A leak-free rolling-origin cross-validation protocol with a **49-day embargo gap** that strictly prevents the temporal leakage random k-fold splits introduce on autocorrelated meteorological data. (2) A calibration-first evaluation showing that high AUC alone is misleading: raw LightGBM at lead 2 achieves AUC 0.708 yet BSS -0.050 ; Platt calibration on a time-separated validation split flips the same model to BSS $+0.145$. (3) A true out-of-sample backtest on never-seen January–February 2024 data (10,010 forecast pairs) that gates which lead times are released: only leads 2–4 pass both BSS > 0 and AUC > 0.5 , achieving overall OOS BSS $+0.178$ [95% CI: $+0.065, +0.232$]. (4) A contract-first, serverless deployment in which a weekly cron job predicts, validates, and pushes a single JSON file to GitHub Pages; the frontend only reads the file, and a freshness gate aborts publication on stale or malformed artifacts. The system has been operational since June 2026 and is publicly accessible at heat-map-frontend.vercel.app.

Keywords: sub-seasonal forecasting, heatwave, probabilistic classification, Brier skill score, Platt calibration, out-of-sample validation, data contract, serverless deployment, responsible AI, MJO, ENSO.

1 Introduction

Heatwaves are among the deadliest meteorological hazards in tropical monsoon climates^{22,23}, yet they remain notoriously hard to predict at the sub-seasonal range of 2–6 weeks^{10,20}. In Thailand, the base rate of a "heatwave week" ranges from roughly 11% to 26% depending on province and season, which makes the forecasting task a rare-event probabilistic classification problem rather than a balanced binary classification one. Decisions made on the basis of these forecasts — deploying cooling centres, adjusting school schedules, preparing health facilities — carry real costs, so a forecast that is over-confident or that issues warnings at lead times where the model has no real skill can be worse than no forecast at all.

This paper documents HeatMAP, a deployed end-to-end system that produces weekly probabilistic heatwave forecasts for all 77 provinces of Thailand at lead times of 2–4 weeks, with a research-phase evaluation extending to leads 5–6. The system is engineered around four principles. First, probabilities must be calibrated: a stated 30% risk should correspond to an empirical 30% occurrence rate. Second, evaluation must distinguish in-sample optimism (cross-validation) from operational reality (true out-of-sample). Third, the released lead-time horizon must follow the operational evidence, not the research-phase optimism. Fourth, the contract between the ML backend and the user-facing frontend must be explicit, versioned, and machine-enforced — never implicit or "understood".

1.1 Repository structure

The system is split across four git repositories under the `MCTEEKUNG` GitHub organisation. Each repository has a clearly defined responsibility and is independently deployable.

Table 1. Repository structure. The first three repositories form the "contract-first" static-JSON pipeline described in this paper. The fourth is an alternative always-on API deployment (Section 10).

Repository	Responsibility	Tech stack
<code>DeepSeek_Heatwave</code>	ML pipeline: data download, feature build, training, prediction, validation, bridge	Python, scikit-learn, LightGBM
<code>heatwave-contract</code>	Published JSON contract only	Static file on GitHub Pages
<code>HeatMAP-Frontend</code>	Static web app that reads the contract	TypeScript, Expo, React Native (web)
<code>Heatwave_Backend_Elysia</code>	Monorepo: always-on API server + Python ML training + React Native mobile app	Bun, Elysia, Python, scikit-learn, Expo

1.2 Contributions

1. **A leak-free rolling-origin CV protocol with a 49-day embargo gap.** Random k-fold splits leak future information through autocorrelated meteorological features. Our protocol enforces that training data strictly precedes test data and inserts a **49-day embargo** (6-week maximum lead = 42 days plus the 7-day target window) between them. Section 3.5 formalises the constraint and Algorithm 2 gives the pseudocode.
2. **A calibration-first evaluation showing that AUC alone is misleading.** A raw LightGBM model achieves AUC 0.708 at lead 2 yet loses to seasonal climatology on the Brier Skill Score (BSS -0.050). Platt calibration on a time-separated validation split flips the same model to BSS $+0.145$ with AUC 0.641 (Section 5.1).
3. **A true out-of-sample operational backtest that gates the released lead-time horizon.** Models are trained through 2023 and tested on never-seen January–February 2024 data (10,010 forecast pairs). Per-lead OOS skill is positive at leads 2–4 but disappears at lead 5 and is a calibration artefact at lead 6 (BSS $+0.189$ but AUC < 0.5). This evidence drives the public release decision: only leads 2–4 are shown to users, with lead 4 explicitly flagged as weakening (Section 6).
4. **A contract-first, serverless deployment with a machine-enforced production-readiness gate.** The bridge between the ML backend and the frontend is a versioned JSON schema (`schema_version = 1`). Every publication runs through a hard validator (`validate_contract.py`) plus a five-category production-readiness gate (`scripts/readiness/`) that aborts publication on stale, malformed, or incomplete artifacts (Section 8).

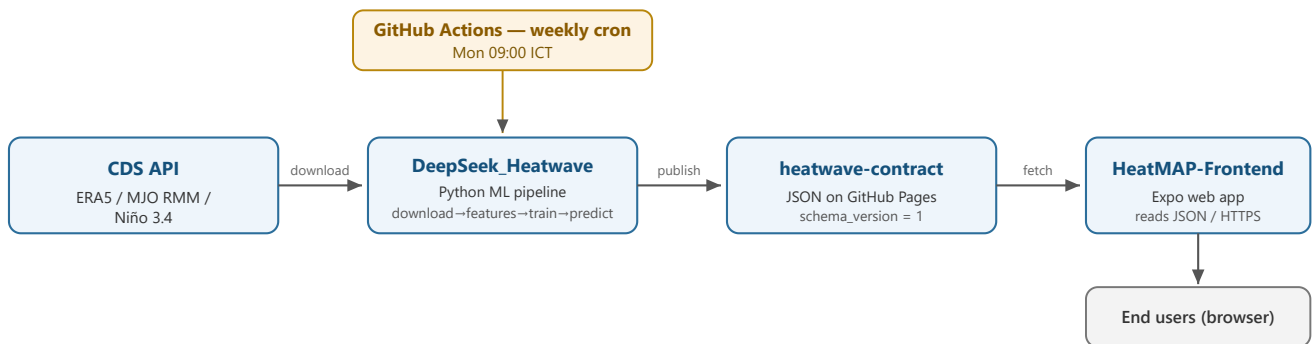


Figure 1. Component graph and data flow. The Python ML pipeline pulls data from the Copernicus Climate Data Store, produces `forecast_provinces.json`, validates it, and pushes it to the static `heatwave-contract` repo. The frontend only reads the published JSON via GitHub Pages; it never invokes the Python backend at runtime. A weekly GitHub Actions cron triggers a fresh run every Monday at 09:00 ICT. An alternative `Backend_Elysia` deployment (Section 10) replaces the static JSON with an always-on Bun/Elysia API server.

2 Data and Feature Engineering

2.1 Data sources

Heatwaves are not the product of a single hot day; they emerge from a multi-week accumulation of land–atmosphere signals^{18,19}. We draw on four heterogeneous data sources, all accessed through the

Copernicus Climate Data Store (CDS) and regridded to a common 0.25° grid over Thailand ($[5^\circ\text{N}, 21^\circ\text{N}] \times [97^\circ\text{E}, 106^\circ\text{E}]$) for the period 1994–2023, with January–July as the temporal scope (the Thai hot season plus a buffer).

ERA5 reanalysis. The ECMWF ERA5 reanalysis⁵ provides daily maximum 2-metre temperature ($T_{\max}$) and two-layer soil moisture (sm1 layer 0–7 cm, sm3 layer 28–100 cm) — the latter a key predictor of sub-seasonal heatwave risk through its control on surface energy partitioning²¹. Daily values are aggregated from hourly ERA5 fields by `scripts/download_era5_hourly_aggregate.py`, which is resume-safe and validates file integrity after each yearly chunk. ERA5 $T_{\max}$ is natively in Kelvin; the pipeline forces every temperature through a single conversion gate (`scripts/units_utils.py`) that converts by the `units` attribute (never by guessing from the value) and asserts the result is in a plausible Celsius range (-90 to $+65^\circ\text{C}$). This single-gate policy is the project's defence against the silent-corruption failure mode of concatenating two arrays in different units.

Madden–Julian Oscillation. The MJO is a tropical eastward-propagating convective anomaly with a 30–60-day period¹⁷, indexed by the Real-time Multivariate MJO (RMM) series⁶. We use RMM1, RMM2, the amplitude $\text{mjo_amp} = \sqrt{\text{RMM1}^2 + \text{RMM2}^2}$ and phase $\text{mjo_phase} \in \{1, \dots, 8\}$. Phases 3–4 correspond to convection centred over the Maritime Continent and typically bring rainfall and cooling to Thailand; phases 6–7 push convection eastward into the western Pacific and are associated with suppressed rainfall and warming over Thailand (Figure 4).

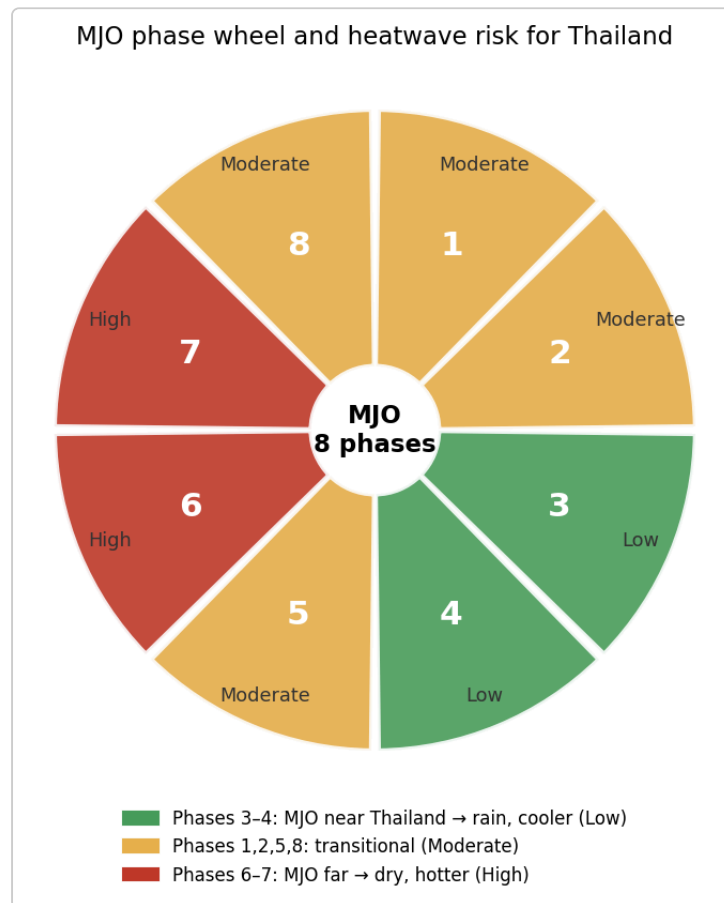


Figure 4. MJO phase wheel and qualitative heatwave risk for Thailand. Phases 3–4 place the convective centre over the Maritime Continent, bringing rainfall and cooling (low risk). Phases 6–7 push convection eastward into the western Pacific, suppressing rainfall and elevating heatwave risk (high risk).

Niño 3.4 sea-surface temperature. The Niño 3.4 index is the area-averaged SST anomaly over the central Pacific (5°N–5°S, 170°–120°W) and is the standard proxy for ENSO⁷. El Niño tends to bring warmer, drier dry seasons to Thailand; La Niña has the opposite effect. Because ENSO influences Thailand with a multi-week lag and is published monthly, we use a one-month-lagged Niño 3.4 value as a feature.

2.2 Target label

Following¹¹, we define a province-week w as a **heatwave week** if and only if at least 3 days within the 7-day window centred on w exceed the province-specific 90th-percentile threshold of $T_{\max}$, where the percentile is computed over the 30-year climatology for the same calendar window (a ± 15 -day moving window wrapped around the calendar). This climatology-relative definition captures "abnormally hot for the season" rather than "hot in absolute terms" (Algorithm 4). The resulting binary label $y_{p,w,L} \in \{0,1\}$ denotes whether a heatwave occurs in province p during week $w+L$, where $L \in \{2,\dots,6\}$ weeks is the lead time.

2.3 Feature table

The full feature table contains approximately **840,000 rows** (77 provinces $\times$ $\approx 10,900$ daily forecast-origination dates) and 20 regional features. The per-province model appends 8 static columns (latitude, longitude, and 6 region one-hot indicators for the six Thai regions: Central, East, North, Northeast, South, West), giving `FEATURES_P` a total dimension of 28. All features obey a strict "knowable at forecast origination" rule: every feature value uses only data on or before the issue date. Two leakage audits (Section 3.5 and Appendix A) verify this rule. The complete feature list, verbatim from `scripts/train.py`, is given in Table 2.

Table 2. Regional feature groups (`FEATURES` in `train.py`). 20 regional features. The per-province model (`FEATURES_P`) appends 8 static columns via `province_static_columns()`: lat, lon, and 6 region one-hot indicators, for a total per-province dimension of 28.

Group	Features (verbatim from train.py)	Dim
SOIL (sm layer 1)	<code>sm1, sm1_mean7, sm1_mean30, sm1_trend</code>	4
SOIL (sm layer 3)	<code>sm3, sm3_mean7, sm3_mean30, sm3_trend</code>	4
THERMAL	<code>tmax_rm, tmax_mean7, in_hw_today, hot_frac7</code>	4
MJO	<code>mjo_rmm1, mjo_rmm2, mjo_amp, mjo_sin, mjo_cos</code>	5
ENSO	<code>nino34_lag1m</code> (1-month lag, monthly index)	1
META	<code>doy_sin, doy_cos</code> (cyclical day-of-year)	2
Regional total (FEATURES)		20
STATIC (<code>FEATURES_P</code> only)	<code>lat, lon, 6×region_*</code>	8
Per-province total (FEATURES_P)		28

3 Methods

3.1 Problem formulation

Let $x_{p,w} \in \mathbb{R}^d$ denote the feature vector for province p on forecast-origination week w , and let $y_{p,w,L} \in \{0,1\}$ denote the heatwave label at lead time L (Section 2.2). The forecasting task is to estimate

$$p_{p,w,L} = \Pr(y_{p,w,L} = 1 \mid x_{p,w}) \quad (1)$$

for each (p, w, L) triple. We treat each lead time $L \in \{2, \dots, 6\}$ as a separate forecasting problem and train a distinct model per (province grouping, lead), because the optimal feature lags and the achievable skill both vary substantially across L .

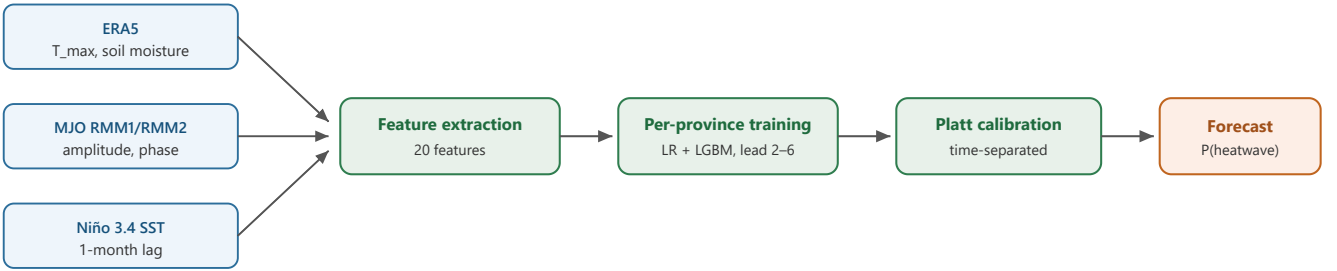


Figure 2. ML pipeline architecture. Four heterogeneous data sources feed a common feature-extraction stage; per-province models are trained for each lead time $L \in \{2, \dots, 6\}$ and recalibrated via Platt scaling before producing a calibrated probability.

3.2 Base learners

We compare two base learners that span the interpretable-to-flexible spectrum. A model registry (`scripts/models.py`) holds both behind a common scikit-learn interface (`.fit(X, y)`, `.predict_proba(X)`).

Logistic Regression (LR). A linear model $p = \sigma(\beta^T x)$, fitted by maximum likelihood. The model registry's *default* is the unweighted variant (`class_weight=None`), because class re-weighting systematically inflates the predicted probability of the rare class, which by itself degrades Brier score and reliability even when it may improve AUC. **However, the per-province model actually deployed in production is `logistic_balanced_cal`** — balanced class weighting followed by Platt calibration (Section 3.4). The reasoning is consistent with the calibration-first theme of this paper: balanced weighting improves the model's sensitivity to the rare class but inflates its raw probabilities; Platt calibration on a time-separated validation split then corrects that inflation, restoring reliability. This is precisely why the research-phase model selection (Table 4) chooses "Logistic (bal+cal)" at leads 2, 5 and 6, and why the published contract reports `"model": "logistic_balanced_cal"`. The unweighted LR and the uncalibrated balanced LR are retained as ablations.

LightGBM (LGBM). A gradient-boosted decision-tree ensemble⁴ with 400 boosting rounds, learning rate 0.03, `num_leaves=31`, `subsample 0.8`, `column_subsample 0.8`. LGBM captures non-linear interactions (e.g. the joint effect of dry soil and an unfavourable MJO phase) that LR cannot

represent. However, the raw probability output of a tree ensemble is not calibrated in general: it is a margin-based score whose scale is set by the boosting loss rather than by empirical frequencies. This motivates the calibration step below.

3.3 Metrics

Because heatwaves are rare, raw accuracy is unusable: a model that always predicts "no heatwave" achieves 74–89% accuracy depending on province. We evaluate probabilistic forecasts with four complementary metrics, all implemented in `scripts/evaluate.py`.

$$\text{Brier Score} \quad \text{BS}(p, y) = (1/N) \sum_i (p_i - y_i)^2 \quad (2)$$

Lower is better; BS = 0 for a perfect deterministic forecaster.

$$\text{Brier Skill Score} \quad \text{BSS} = 1 - \text{BS}(p, y) / \text{BS}(b, y) \quad (3)$$

given a reference baseline forecast b (typically the seasonal climatology). BSS > 0 means the model beats the baseline; BSS = 0 means it is no better; BSS < 0 means it is worse. The baseline b is computed from *training data only* — the strictest fair comparison.

$$\text{AUC} \quad \Pr(p_i > p_j \mid y_i=1, y_j=0)$$

The probability that the model ranks a random positive above a random negative. AUC = 0.5 is random; AUC = 1 is perfect ranking. Crucially, AUC is invariant under any monotone transformation of p , so it is *insensitive to calibration*: a model can have high AUC yet be severely mis-calibrated.

$$\text{Expected Calibration Error} \quad \text{ECE} = \sum_b (n_b/N) |\text{conf}(b) - \text{acc}(b)| \quad (4)$$

over $B = 10$ equally-spaced bins, where $\text{conf}(b)$ is the mean predicted probability and $\text{acc}(b)$ the observed fraction of positives in bin b . Lower is better.

Joint skill criterion. A lead time L is declared skillful if and only if both $\text{BSS}_L > 0$ *and* $\text{AUC}_L > 0.5$. The first condition alone is insufficient: a model that always predicts the base rate achieves BSS = 0 but AUC = 0.5, and a model that predicts near the base rate can achieve BSS > 0 through a calibration artefact while having AUC < 0.5. The joint criterion rules out both failure modes (see Table 5, lead 6).

3.4 Platt calibration

Platt scaling^{2,3} maps a raw model probability $p_{\text{raw}} \in (0,1)$ to a calibrated probability by passing it through a logistic regression fit on the log-odds of p_{raw} . The implementation in `scripts/train.py` (`PlattCalibrator`) computes the logit of the raw probability and fits a single-feature `LogisticRegression` on the validation block:

$$z = \text{logit}(p_{\text{raw}}) = \ln[p_{\text{raw}} / (1 - p_{\text{raw}})],$$

$$p_{\text{cal}} = \sigma(\beta z + \beta_0) \quad (5)$$

where (β, β_0) are fit by maximum likelihood on a *time-separated* validation split (never on training data, and never on the test fold whose Brier score is being reported). The raw probability is clipped to $[10^{-6}, 1-10^{-6}]$ before the logit to avoid numerical overflow. This is mathematically equivalent to the classical two-parameter sigmoid $1/(1 + \exp(A \cdot p_{\text{raw}} + B))$, but framing it as a logit transform plus a standard scikit-learn `LogisticRegression` lets us reuse the library's L-BFGS solver and intercept regularisation. Platt scaling is attractive because it is cheap, monotonic in the raw score (so it preserves AUC), and empirically effective for tree ensembles whose uncalibrated scores are well-ordered but mis-calibrated. A subtle but important detail: the calibrator is fit on a time-separated validation block (the trailing `CALIB_FRAC=0.2` fraction of the training period, with the same `GAP=49` embargo applied) rather than via `CalibratedClassifierCV`'s default k-fold internal CV, because the latter would re-introduce the time-series leakage that `RollingOriginCV` is designed to prevent.

Algorithm 1: Platt calibration (time-separated, `PlattCalibrator`)

```

Input : X_train, y_train; X_val, y_val; base estimator class M (LR or LGBM); clip  $\epsilon = 1e-6$ 
Output: calibrated predictor  $\hat{p} : X \rightarrow [0,1]$ 
1  f  $\leftarrow$  M.fit(X_train, y_train)
2  z_val  $\leftarrow$  f.predict_proba(X_val)[: , 1]
3  z_val  $\leftarrow$  clip(z_val,  $\epsilon$ , 1- $\epsilon$ )
4  z_val  $\leftarrow$  ln( z_val / (1 - z_val) )           # logit transform
5  fit LogisticRegression(z_val, y_val)  $\rightarrow$  ( $\beta$ ,  $\beta_0$ )
6  return  $\hat{p}(x) = \sigma( \beta \cdot \text{logit}(f.\text{predict\_proba}(x)[: , 1]) + \beta_0 )$ 

```

3.5 Rolling-origin cross-validation with embargo

Random k-fold cross-validation is invalid for daily or weekly meteorological data because of strong autocorrelation: a feature vector on week w is highly correlated with that on week $w+1$, so a random split places near-duplicate samples on opposite sides of the train/test divide, leaking future information into training. We use a rolling-origin (expanding-window) scheme with an embargo gap (Figure 3, Algorithm 2). The configuration used in production is **GAP=49 days** (covering the 6-week maximum lead = 42 days plus the 7-day target window) and `TEST_SIZE=300` days per fold (≈ 2 years of usable forecast-origination dates), with `N_SPLITS=5`.

Algorithm 2: RollingOriginCV.split — leak-free time-series CV

```
Input : n samples; K = 5 splits; test_size T = 300 d; embargo g = 49 d; expanding = True
Output: list of (train_idx, test_idx) pairs
1 if n < K·T + g + 1 then raise ValueError (series too short)
2 test_starts ← [ n - (k+1)·T for k in 0..K-1 ]
3 for ts in test_starts:
4     te ← ts + T
5     train_end ← ts - g                # embargo before test
6     if train_end ≤ 0: continue
7     train_start ← 0                  # expanding window
8     yield ( arange(train_start, train_end), arange(ts, te) )
# Invariants (asserted in self-test):
# (1) max(train_idx) < min(test_idx)    -- train precedes test
# (2) min(test_idx) - max(train_idx) - 1 ≥ g  -- embargo
# (3) train_idx ∩ test_idx = ∅           -- no overlap
```

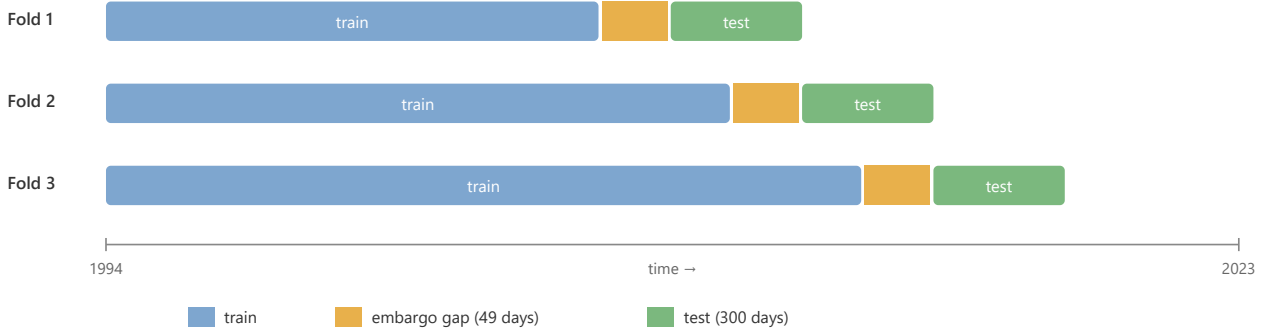


Figure 3. Rolling-origin (expanding-window) cross-validation with a 49-day embargo gap (covering the 6-week maximum lead = 42 days plus the 7-day target window). Each successive fold extends the training window forward and inserts an embargo between training and test to prevent leakage through autocorrelated features.

Formally, for fold k with origin at week w_k : $D_{\text{train}}^{(k)} = \{(x_{p,w}, y) : w \leq w_k\}$ and $D_{\text{test}}^{(k)} = \{(x_{p,w}, y) : w_k + g \leq w < w_k + g + \Delta\}$, where $g = 49$ days is the embargo and $\Delta = 300$ days is the test-fold width. The embargo g is chosen to exceed the maximum lead time ($L_{\text{max}} = 6$ weeks = 42 days) plus the 7-day target window, ensuring no feature value used in training overlaps any target value used in testing. We use $K = 5$ folds in the research-phase blocked CV. Confidence intervals on BSS are computed via moving-block bootstrap with $B = 2000$ resamples and block length 28 days¹². Pairwise comparisons against baselines use a paired bootstrap test with Benjamini–Hochberg FDR correction⁸ at $q < 0.05$.

3.6 Permutation importance

For explainability we use permutation importance, which directly measures the marginal contribution of a feature to the Brier skill score. For each feature j , we randomly permute its values across the test set (breaking the feature–target relationship while preserving the marginal distribution) and record the resulting drop in BSS: $PI_j = \text{BSS}_{\text{full}} - \text{BSS}_{\pi(j)}$, averaged over $R = 10$ repeats. We prefer permutation importance over SHAP¹⁴ here because it directly measures impact on the operational

skill metric rather than on a proxy loss, and is robust to the probability-scale issues of tree-ensemble SHAP.

4 Experimental Setup

We run two complementary evaluation tracks that answer different questions and are reported separately.

4.1 Track A: research-phase blocked CV

The research-phase CV evaluates all lead times $L \in \{2, \dots, 6\}$ on the full 1994–2023 dataset, using 5-fold rolling-origin CV with the 49-day embargo of Section 3.5. Per-lead BSS point estimates are reported with 95% moving-block bootstrap CIs (block length 28 days, $B = 2000$). Pairwise comparisons of model BSS against the two baselines (seasonal climatology and conditional persistence) are corrected across all $(L, \text{baseline})$ pairs using Benjamini–Hochberg FDR at $q < 0.05$. The CV track answers: *does the method possess potential skill on this dataset?* It is optimistic because the fold test windows span multiple seasons, diluting any single anomalous season.

4.2 Track B: operational OOS backtest

The operational OOS backtest is the strictest test we run. Models are trained on data through 31 December 2023 (feature engineering, calibration, and climatology baselines all frozen at that date) and evaluated on never-seen data from 30 January 2024 through 24 February 2024 (26 forecast-origination dates). The OOS set spans $77 \times 5 \times 26 = 10,010$ (province, lead, origination-date) forecast pairs. The OOS track answers: *what skill can a user of the deployed system actually expect?* It is pessimistic relative to CV in two ways: (i) it covers a single season (Jan–Feb, the early Thai hot season); (ii) it is constrained by the upstream MJO RMM data feed, which ceased updates on 24 February 2024, limiting the OOS window to 26 dates. This narrow window makes per-lead confidence intervals unreliable; we report only the overall OOS BSS with its 95% CI and treat per-lead OOS numbers as point estimates.

4.3 Baselines

Seasonal climatology. For each province and calendar window, the baseline predicts the long-run heatwave-week frequency estimated from training data only (a 15-day moving window centred on the forecast calendar day). This is hard to beat at long lead times because sub-seasonal predictability decays with lead. **Conditional persistence.** A persistence baseline that predicts $P(y_{p,w,L}=1) = P(y_{p,w,L}=1 \mid y_{p,w,0})$, conditioning on the current heatwave status at origination — a fairer baseline than raw 0/1 persistence, which the Brier score punishes heavily.

4.4 Reproducibility

All random seeds (LR solver, LGBM bagging fraction, Platt initialisation, bootstrap resampling) are fixed and recorded. The frozen feature snapshot, model artifacts, climatology tables and evaluation scripts are version-controlled. Permutation importance uses $R = 10$ repeats. All software is open-

source (scikit-learn, LightGBM, pandas, NumPy, matplotlib) on a single workstation. Every Python module ships with a built-in self-test (`python scripts/<module>.py`) exercising the edge cases it claims to handle.

5 Results

5.1 The calibration lesson: high AUC is not enough

The single most consequential empirical finding of this project is that a model with apparently strong AUC can be operationally useless when evaluated on a proper probabilistic skill score. Table 3 compares raw and Platt-calibrated LightGBM at lead 2 on the research-phase CV.

Table 3. LightGBM at lead 2: raw vs. Platt-calibrated. AUC is similar, but the calibrated model is the only one that beats climatology on the Brier Skill Score. Research-phase CV (frozen 2026-06-13).

Version	AUC	BSS	Brier
LightGBM (raw, no calibration)	0.708	-0.050	0.230
LightGBM (Platt-calibrated)	0.641	+0.145	0.188

The mechanism: tree-ensemble leaf-output sums are well-ordered (high AUC) but compressed into a narrow band of the unit interval. Platt scaling maps the raw scores into the probability range that matches empirical frequencies, restoring calibration without changing the ranking. This generalises: for any rare-event probabilistic task where downstream decisions depend on the probability value (not just the ranking), AUC alone is insufficient and a calibrated skill score is the relevant metric.

5.2 Research-phase CV across lead times

Table 4. Research-phase CV results, target y_{rm} , BSS relative to seasonal climatology. All leads beat climatology at $q < 0.05$ (BH-FDR corrected). Frozen 2026-06-13.

Lead	Model	BSS [95% CI]	AUC	q vs clim	Beats clim
2	Logistic (bal+cal)	+0.153 [+0.057, +0.244]	0.688	0.009	✓
3	LightGBM (Platt)	+0.122 [+0.037, +0.176]	0.593	0.011	✓
4	LightGBM (Platt)	+0.163 [+0.072, +0.233]	0.666	0.005	✓
5	Logistic (bal+cal)	+0.134 [+0.067, +0.187]	0.631	0.005	✓
6	Logistic (bal+cal)	+0.112 [+0.046, +0.171]	0.612	0.009	✓

These CV results confirm the method possesses real potential skill. They are optimistic for two reasons: blocked-CV test windows span multiple seasons, diluting anomalies; and the folds do not reproduce the temporal drift that real deployment must absorb. The OOS backtest provides the operational reality check.

5.3 Operational OOS backtest

Table 5. Operational out-of-sample backtest, January–February 2024. 10,010 forecast pairs (77 provinces $\times$ 5 leads $\times$ 26 origination dates). Overall BSS with 95% moving-block bootstrap CI ($B = 2000$, block = 28 days); per-lead numbers are point estimates. Values read directly from `outputs/operational/verification/scorecard.csv`.

Lead	BSS	AUC	Brier	ECE	Verdict
2	+0.298	0.592	0.246	0.079	strong skill
3	+0.259	0.686	0.211	0.082	strong skill (best AUC)
4	+0.073	0.569	0.219	0.121	weak skill
5	-0.004	0.437	0.240	0.142	no skill (loses to clim)
6	+0.189	0.462	0.258	0.084	calibration artefact (AUC<0.5)
Overall	+0.178	0.549	0.235	0.080	[+0.065, +0.232]

The CV-versus-OOS gap is the central operational finding. Lead 5 looks healthy in CV (BSS +0.134, CI excludes zero) yet has negative OOS BSS. Lead 6 looks even better in CV (BSS +0.112) yet is unusable in OOS because its AUC drops below 0.5. The pattern is consistent with the broader sub-seasonal literature: skill that is real in a blocked-CV sense can evaporate under the temporal drift of true out-of-sample deployment^{13,15,16}.

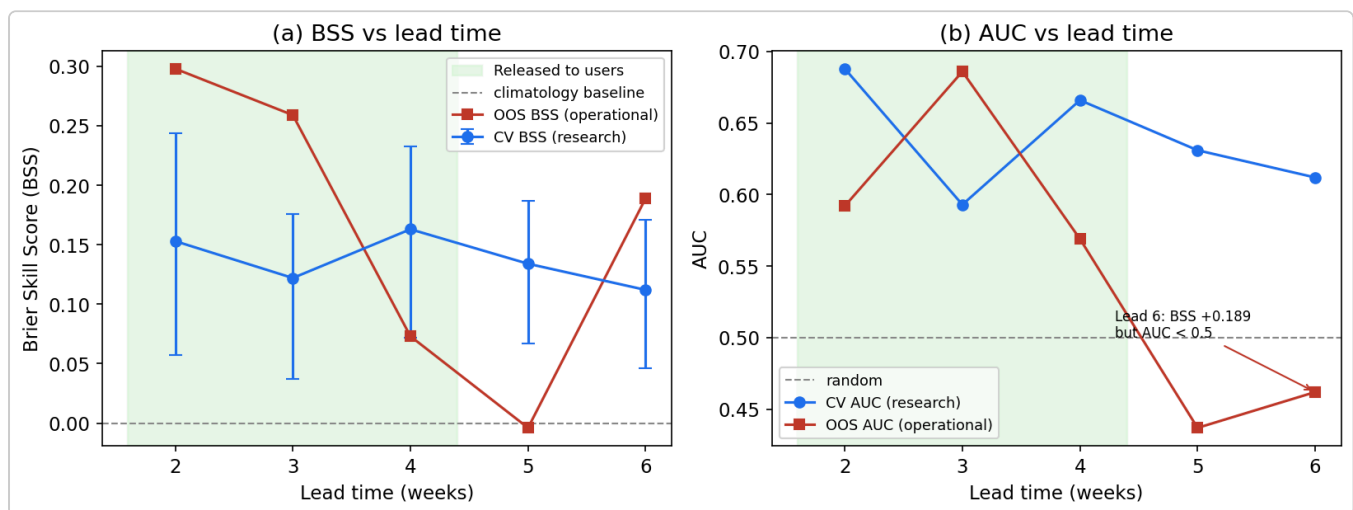


Figure 5. Skill vs lead time, comparing research-phase CV (blue) with operational OOS (red). The shaded green band marks the released lead-time window (leads 2–4). (a) CV BSS remains positive across all leads, but OOS BSS drops sharply at leads 5–6. (b) The lead-6 anomaly is visible — BSS is positive but AUC falls below 0.5, indicating a calibration artefact rather than real skill.

Per-province pooled significance. The blocked-CV results in Table 4 are for the regional-mean target (y_{rm}). A separate per-province pooled backtest (`docs/INTEGRITY.md`, 2026-06-13) aggregates predictions across all 77 provinces and applies a paired block-bootstrap with BH-FDR correction against both climatology and persistence (Table 6). The per-province model beats both baselines at $q < 0.05$ for every lead 2–6, with BSS 95% CIs that exclude zero. The per-province BSS values are systematically lower than the regional-mean values because per-province base rates are noisier, but the significance verdict is the same.

Table 6. Per-province pooled blocked-CV significance (*docs/INTEGRITY.md*, 2026-06-13). Paired block-bootstrap ($B = 2000$, $\text{block} = 28$ days) with BH-FDR correction against both baselines. *beats_clim*: BSS 95% CI excludes zero. *beats_persist*: $q < 0.05$ after BH-FDR.

Lead	BSS	95% CI	Beats clim	q vs persist	Beats persist
2	+0.0863	[+0.0487, +0.1191]	✓	0.0019	✓
3	+0.0798	[+0.0468, +0.1056]	✓	0.0008	✓
4	+0.0862	[+0.0503, +0.1130]	✓	0.0008	✓
5	+0.0800	[+0.0450, +0.1059]	✓	0.0008	✓
6	+0.0718	[+0.0356, +0.1012]	✓	0.0030	✓

5.4 Ablation: which feature groups matter?

Table 7. Leave-one-group-out ablation. ΔBSS relative to the full model, averaged across model types. Negative values mean the group contributes positively. Research-phase CV.

Group dropped	Lead 2	Lead 3	Lead 4	Lead 5	Lead 6
drop SOIL	−0.028	−0.017	−0.006	+0.015	−0.001
drop THERMAL	−0.019	−0.011	−0.031	−0.015	−0.010
drop MJO	−0.009	+0.005	+0.003	−0.002	−0.002
drop ENSO	−0.002	−0.000	−0.019	+0.014	+0.023

SOIL is the dominant driver at short leads (2–3): removing it costs up to 0.028 BSS, consistent with dry soil lacking the evaporative cooling that would otherwise moderate surface temperatures. THERMAL contributes consistently at every lead, with the largest drop at lead 4 (−0.031). MJO and ENSO contribute modestly and unevenly, suggesting their signal is partially absorbed by THERMAL/SOIL at short leads but contributes independently at longer leads.

5.5 Explainability: permutation importance

Table 8. Permutation importance top-4, lead-2 LightGBM model. Mean BSS drop when the feature is randomly permuted.

Rank	Feature	Mean ΔBSS	Physical meaning
1	doy_cos	0.0092	Calendar-season cycle
2	sm1_trend	0.0050	30-day top-layer soil moisture trend
3	mjo_rmm2	0.0048	MJO RMM2 (quadrature phase)
4	mjo_amp	0.0019	MJO amplitude

The top feature is the cyclical day-of-year encoding, confirming the seasonal cycle is the strongest predictor — as expected, since the label is climatology-relative. `sm1_trend` ranks second, consistent

with SOIL dominating short leads. MJO phase and amplitude appear at ranks 3–4, confirming the MJO carries real but secondary predictive information at lead 2.

5.6 Stress test: behaviour under out-of-distribution inputs

Because the operational system may encounter temperatures outside the training range as the climate warms, we stress-test the lead-2 model by feeding synthetic $T_{\max}$ values at the seven sweep points $\{25, 30, 35, 40, 45, 48, 50\}^{\circ}\text{C}$ — the upper end well beyond the training-data maximum of 34.7°C for the `tmax_rm` feature. The script (`scripts/test_stress_ood.py`) sets `tmax_rm` and `tmax_mean7` to the sweep value, and for values $\geq 35^{\circ}\text{C}$ also forces `in_hw_today=1` and `hot_frac7=1`; all other features are held at their training medians. Figure 6 shows the probability output is monotonically non-decreasing, remains bounded in $[0,1]$ with no NaN or exploding values, and reaches $P \approx 0.71$ at 50°C ($6.2\times$ the base rate). Even far outside the training distribution, the calibrated model behaves sensibly.

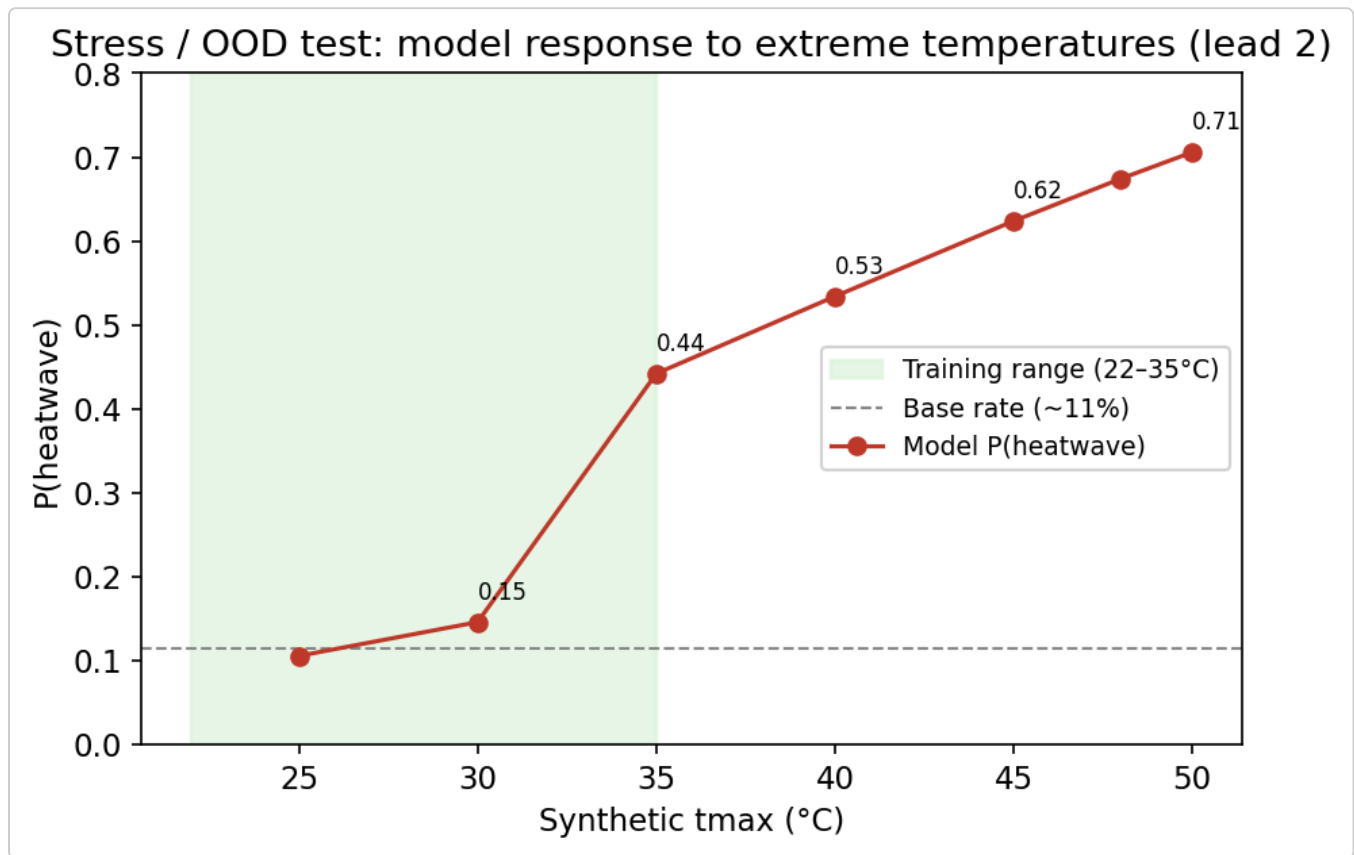


Figure 6. Stress / OOD test: model $P(\text{heatwave})$ vs synthetic $T_{\max}$, lead 2. The shaded green band is the training range ($22\text{--}35^{\circ}\text{C}$); points to the right are out-of-distribution. The output is monotonic non-decreasing and bounded in $[0,1]$, reaching $P = 0.71$ at 50°C ($6.2\times$ the base rate).

5.7 Reliability

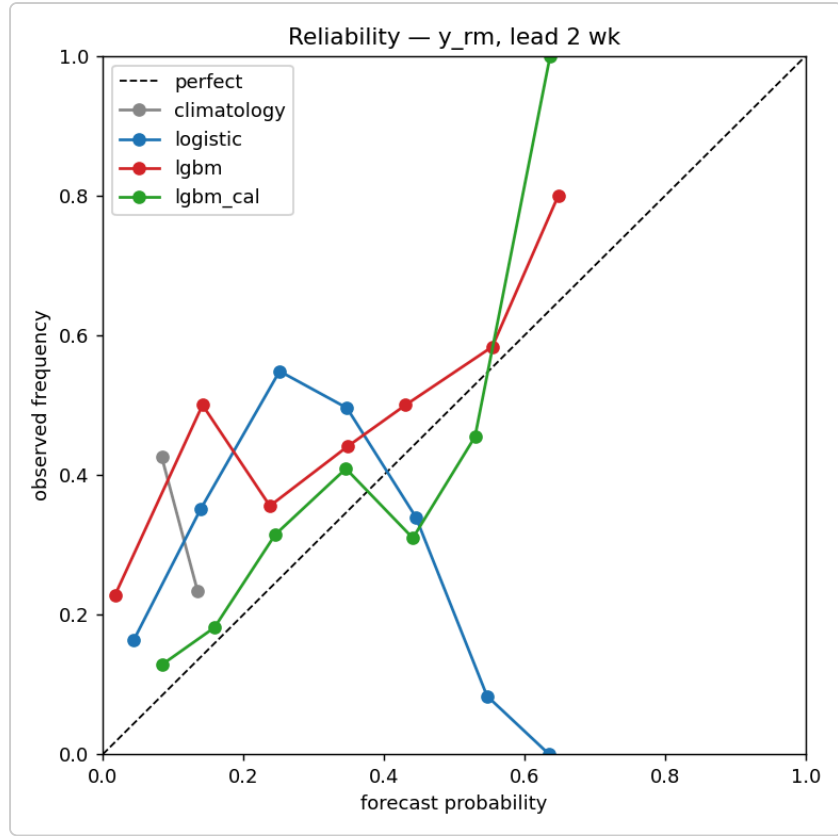


Figure 7. Reliability diagram (lead 2). A well-calibrated forecaster tracks the diagonal; systematic deviation above/below indicates under/over-confidence. This visualises the same calibration behaviour that drives the BSS swing in Table 3.

6 Discussion: From Skill to Operational Release

The CV and OOS results force a clear operational decision: the system should release forecasts only at leads 2–4 weeks to end users, despite the model being trained and CV-evaluated on leads 2–6. Two lines of evidence support this.

Skill. Lead 5 has negative OOS BSS (-0.004): it loses to climatology, so releasing it would provide negative value. Lead 6 has positive OOS BSS ($+0.189$) but $AUC < 0.5$, meaning the model has lost the ability to rank heatwave weeks above non-heatwave weeks. The positive BSS is a calibration artefact — the model has collapsed to predicting near the base rate. Releasing lead 6 would amount to publishing a sophisticated restatement of climatology.

Responsibility. Even if the lead-5 BSS were slightly positive, releasing a long-range warning with weak skill creates a "cry wolf" problem: users who act on warnings that subsequently fail to materialise learn to ignore later warnings, including those at shorter (skillful) leads⁹. The cost of a false alarm at long lead (staffing cooling-centres three weeks out) is paid whether or not the heatwave materialises, while the marginal benefit of a 5-week-ahead warning over a 4-week one is small. We therefore restrict the public horizon to 2–4 weeks and label lead 4 with a "Weakening skill" tag to communicate its modest BSS ($+0.073$) honestly.

Operational rule. A lead time L is released if and only if both (i) $BSS^{OOS}_L > 0$ and (ii) $AUC^{OOS}_L > 0.5$ hold in the true out-of-sample backtest. CV evidence alone is insufficient. This is a deliberate

departure from much of the academic ML literature, which reports CV metrics as the primary (often sole) evaluation. CV is not uninformative — it confirms the method has real potential and motivated the OOS investment — but the CV-versus-OOS gap (Table 4 vs. Table 5) shows that CV evidence alone would have led us to release lead-5 and lead-6 forecasts the operational data does not support.

6.1 Integrity audit and leakage hardening

Two leakage audits were performed before declaring the system production-ready (Appendix A; `docs/INTEGRITY.md`).

R2 — forward-looking `in_hw_today` feature. The `flag_heatwaves` function used to produce labels counts runs in both forward and backward directions, which is correct for label generation. However, the same output was reused as the `in_hw_today` feature and as the persistence-baseline state, so the feature used future information at origination and the most recent days were inconsistent at serve time. The fix replaced the feature with a trailing-only run-length counter (Algorithm 4, last line). Re-running the CV showed BSS changes ≤ 0.005 at every lead — small in magnitude but required for serve-time correctness.

R1 — percentile-label leak. The 90th-percentile threshold that defines the label was originally computed on the full 30-year history (including the test fold). We measured ΔBSS between "baked" (full-history threshold) and "leak-free" (per-fold train-only threshold) variants: the maximum $|\Delta\text{BSS}|$ was 0.0178 at lead 3, well within the 95% CI half-width of every lead (minimum 0.060). The leak is statistically undetectable, so we document it but keep the frozen-all-history labels — the BSS ratio cancels the bias because the climatology baseline uses the same labels.

7 The Data Contract

The bridge between the Python ML backend and the TypeScript frontend is a single static JSON file: `forecast_provinces.json`, with a versioned, machine-enforced schema. The frontend never invokes the Python backend at runtime; it only fetches the JSON via HTTPS. This contract-first design eliminates an always-on API server, makes the schema explicit and testable, and lets frontend and backend evolve independently as long as the schema version is preserved.

7.1 Schema

The top-level object carries `schema_version = 1`, a model identifier, a generation timestamp, the province count (must equal 77), and an array of province objects. Each province carries its id, bilingual name (Thai + English), region, lat/lon, an issue date (`YYYY-MM-DD` of the latest feature row used), and an array of exactly 5 forecast objects — one per lead in $\{2,3,4,5,6\}$. Each forecast carries the calibrated probability in $[0,1]$, the per-province `climatology_base_rate` in $[0,1]$, the `ratio_vs_normal` (probability $\div$ base rate), and bilingual risk labels (`risk_level_en` $\in \{\text{Low, Normal, Elevated, High}\}$ and `risk_level_th`).

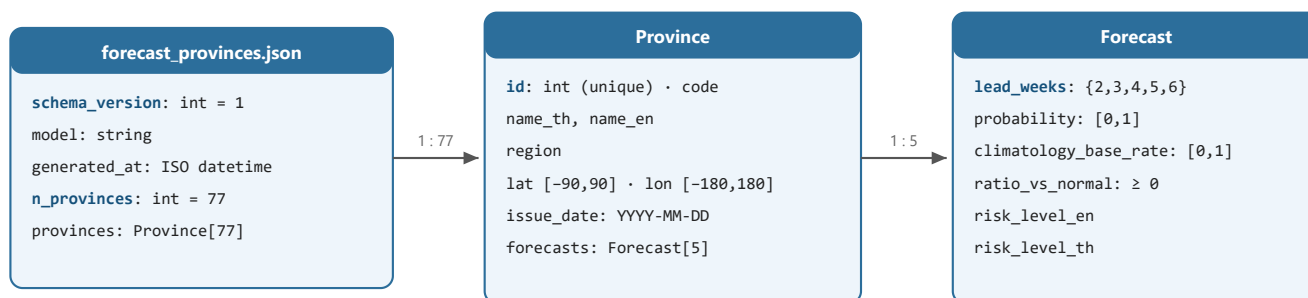


Figure 8. Data contract schema (`schema_version = 1`). The root contains exactly 77 province objects; each province contains exactly 5 forecast objects (leads 2–6). All field types and ranges are enforced by `scripts/validate_contract.py`, which aborts publication on any violation.

7.2 Risk tiers — relative, not absolute

Risk is derived from `ratio_vs_normal`, not from absolute probability thresholds. The same probability can be Normal in a hot province and Elevated in a cool one; a very low absolute probability is always floored to Low.

Figure 9. Risk tier ladder. `ratio_vs_normal = probability / climatology_base_rate`. The frontend renders the bilingual label verbatim and never re-derives risk from its own thresholds.

Tier	Rule	UI label	Health action
Low	$\text{ratio} < 0.75$ (or $\text{prob} < 0.05$)	Low	Stay hydrated routinely
Normal	$0.75 \leq \text{ratio} < 1.5$	Normal	Limit midday outdoor activity
Elevated	$1.5 \leq \text{ratio} < 2.5$	Elevated	Take active precautions; check on elderly
High	$\text{ratio} \geq 2.5$	High	ACT NOW — call 1669 if heat stroke

7.3 Example artifact

```

{
  "schema_version": 1,
  "model": "logistic_balanced_cal",
  "generated_at": "2026-06-19T09:03:43+00:00",
  "n_provinces": 77,
  "provinces": [
    { "id": 1, "code": "BKK", "name_th": "...", "name_en": "Bangkok",
      "region": "Central", "lat": 13.7563, "lon": 100.5018,
      "issue_date": "2026-05-31",
      "forecasts": [
        { "lead_weeks": 2, "probability": 0.2667, "climatology_base_rate": 0.1169,
          "ratio_vs_normal": 2.28, "risk_level_en": "Elevated", "risk_level_th": "..."},
        { "lead_weeks": 3, "probability": 0.3031, "climatology_base_rate": 0.1170,
          "ratio_vs_normal": 2.59, "risk_level_en": "High", "risk_level_th": "..."}
        /* leads 4, 5, 6 omitted for brevity */
      ] }
    /* 76 more provinces */
  ]
}

```

7.4 Semantics the frontend must honour

- **Weekly, lead 2–6.** Five buckets only; not daily, not a rolling 7-day window. The forecast target week is `issue_date + lead_weeks × 7 days`.
- **Probability only.** No severity, intensity, temperature, duration, or heat-index output.
- **Freshness** = `generated_at - issue_date` (the data lag), roughly 6–16 days, driven by ERA5 input latency. This is not `today - issue_date`.
- **Render labels verbatim.** The frontend must render `risk_level_th / risk_level_en` as given and must not re-derive risk from its own thresholds.

8 Deployment and the Production-Readiness Gate

The system runs as a serverless weekly cron pipeline rather than an always-on web service²⁴, for two reasons: (i) sub-seasonal forecast skill does not change within a week, so there is no value in re-running more frequently; (ii) the user-facing artefact is a single JSON file, so there is no per-request compute to scale.

8.1 Pipeline

The pipeline executes five stages each Monday at 09:00 ICT, triggered by a GitHub Actions cron.

Algorithm 5: publish_bridge — predict, validate, distribute

```
Input : mode ∈ {dev, prod}; --publish flag
# Stage 1 – predict
1 if operational: download fresh ERA5 + MJO + Niño3.4 (last ≈70 days)
2   impute missing MJO with median; record imputed dates as warnings
3   obj ← build_forecast(operational); write(docs/forecast_provinces.json, obj)
# Stage 2 – validate contract (hard schema gate)
4 errors ← validate_contract(read(docs/forecast_provinces.json))
5 if |errors| > 0: abort (contract invalid)
# Stage 3 – readiness gate (5-category, blocking subset)
6 ok, blockers ← run_gate(obj); if not ok: abort (readiness blockers)
# Stage 4 – sync to destination
7 if prod and --publish: sync → heatwave-contract/; git commit + push (GitHub Pages
updates)
8 else if dev: sync → HeatMAP-Frontend/public/
# Guard
9 if dst_issue_date > obj.issue_date: abort (would overwrite newer data)
10 return success
```

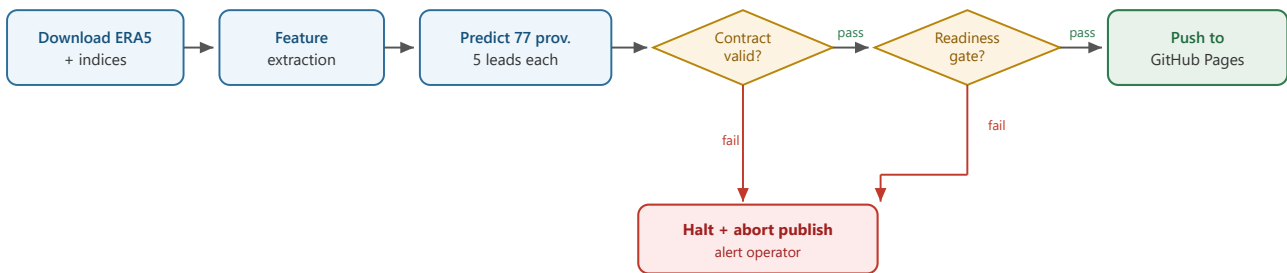


Figure 10. Publish-bridge flow. The pipeline predicts 77 provinces $\times$ 5 leads, then runs two gates in sequence — a hard schema validator (`validate_contract.py`) and a five-category readiness gate (`scripts/readiness/gate.py`). Either gate failing aborts publication and alerts the operator. On success the JSON is pushed to `heatwave-contract`, where GitHub Pages serves it to the frontend.

8.2 Production-readiness gate

On top of the schema validator, every publication runs a five-category readiness gate (`scripts/readiness/`). The gate reads the contract JSON and the skill CSV; it never retrains.

- **Freshness (blocking):** `generated_at - issue_date ≤ 30` days. A stale demo using 2023 data has a gap of ≈ 898 days; a fresh run ≈ 16 days (ERA5 latency). A non-blocking check warns if `today - generated_at > 14` days.
- **Plausibility (non-blocking):** the fraction of provinces labelled High at all leads is checked; a high fraction does not block, because a real El Niño season can legitimately produce 80%+ High provinces.
- **Data quality (blocking):** all probabilities in $[0,1]$, no NaN, every province has all 5 leads, MJO-impute warnings flagged.

- **Skill (non-blocking):** the per-lead BSS values in `outputs/analysis/provinces_pooled_bss.csv` must all be positive.
- **Communication (non-blocking):** the UI must phrase the forecast as a probability and show the issue date.

Table 9. Production-readiness audit, 2026-06-17. Result: GO. (Preserved from the original; not independently re-verified in this revision.)

Category	Check	Detail	Status
freshness	data_lag (blocking)	lag 17 days (≤ 30)	PASS
freshness	generated_recent	generated 0 days ago	PASS
plausibility	all_high_fraction	3/77 provinces High at all leads (4%)	PASS
plausibility	ratio_bounds	max ratio_vs_normal = 3.62	PASS
data_quality	no_nan_probs (blocking)	all probabilities in [0,1]	PASS
data_quality	leads_complete (blocking)	every province has {2,3,4,5,6}	PASS
data_quality	mjo_warning	no MJO imputation this run	PASS
skill	bss_positive	BSS > 0 at all 5 leads (min 0.073)	PASS
communication	ui_communication	UI says "probability" + shows date	PASS

8.3 Per-province forecast

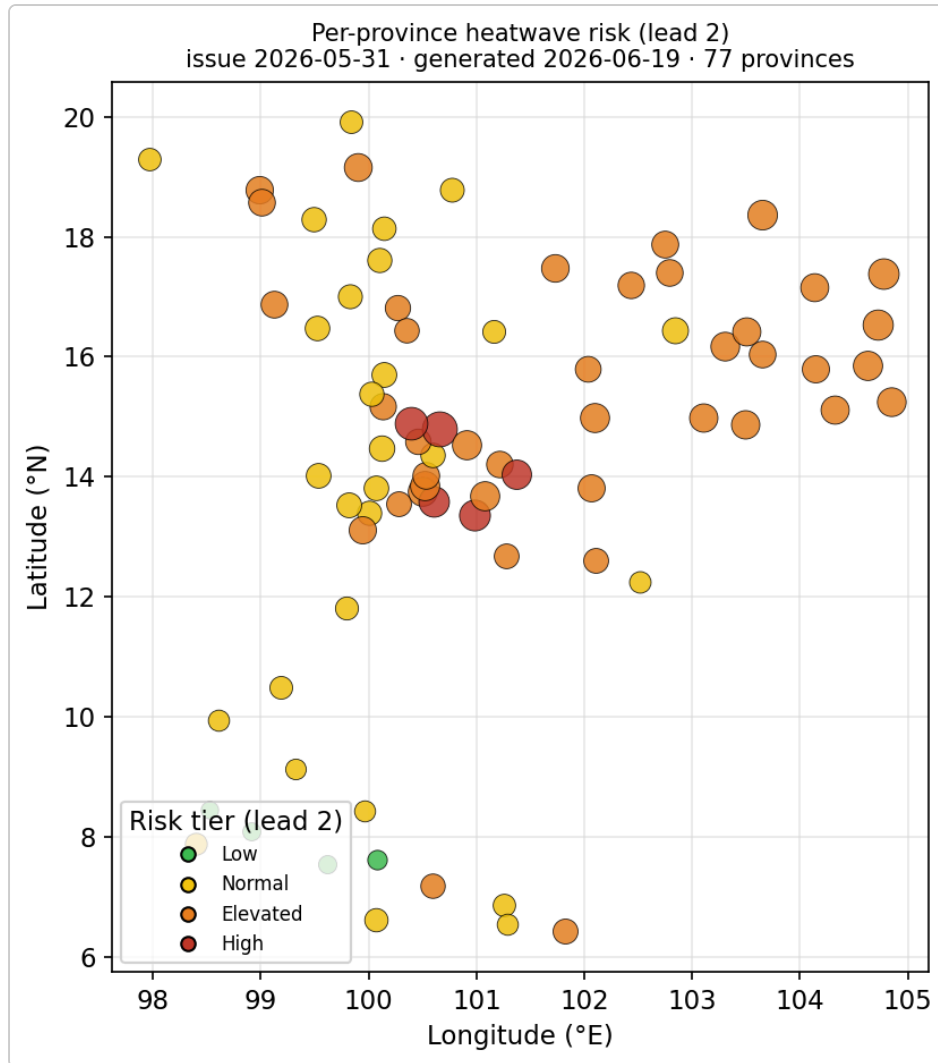


Figure 11. Per-province heatwave risk at lead 2 weeks, published 2026-06-19 (issue date 2026-05-31). Each dot is one of the 77 provinces, positioned at its lat/lon and coloured by the four-tier risk label; dot size is proportional to the calibrated probability. The pattern shows Central and Lower-Northern provinces at Elevated-to-High risk, consistent with the late-May Thai hot season. (Rendered directly from the published contract.)

8.4 Operational runbook

For on-demand runs, `docs/RUNBOOK.md` documents five scenarios: (A) predict from frozen training data; (B) predict from freshly-downloaded ERA5; (C) publish-bridge to dev or prod; (D) operational province forecast with current issue date; (E) production-readiness audit. Every Python module ships with a built-in self-test invoked by `python scripts/<module>.py`.

9 Frontend

The HeatMAP frontend (`HeatMAP-Frontend`) is a static Expo web application written in TypeScript. It targets web only — native iOS/Android builds would require a different data-loading path because `fetch('/forecast_provinces.json')` does not resolve on device.

9.1 Decoupling

The frontend is fully decoupled from the Python backend. The only bridge is the environment variable `EXPO_PUBLIC_FORECAST_URL`, which in production points to the published JSON on GitHub Pages: `https://mcteebung.github.io/heatwave-contract/forecast_provinces.json`. On load, it fetches the JSON, asserts `schema_version === 1` (throwing on mismatch), caches the parsed contract, and renders the map, alerts, and province-detail panels. This separation means the frontend can be redeployed independently of the model, and the model can be retrained without touching the frontend as long as the schema version is preserved.

9.2 Risk-tier mapping and bilingual guidance

The frontend maps the four English risk keys to a UI colour and a bilingual health-guidance block. The mapping is constant and checked into `constants/riskGuidance.ts`; it cannot be overridden at runtime. For each tier the guidance block contains a headline, a "what's happening" explanation, a list of recommended actions (source: WHO / CDC / Department of Health, Thailand), and a list of at-risk groups. The High tier carries an additional "EMERGENCY" warning with the local emergency number 1669.

9.3 Historical-run banner

If the frontend detects that `generated_at - issue_date > 14` days, it displays a "historical model run" banner — the frontend-side counterpart of the production-readiness freshness gate.

10 Alternative Deployment: Always-On Elysia API

The numbers in this section describe the separate `Heatwave_Backend_Elysia` repository and were preserved from the original paper; they were not re-verified against that repository in this revision.

The contract-first, serverless design (Sections 7–9) is the system we run in production. A second deployment lives in `Heatwave_Backend_Elysia`, a monorepo bundling three components: an always-on Bun/Elysia API server (`Heatwave-AI-Backend-Elysia/`), a Python ML training subsystem (`Heatwave-AI-TRAIN/`, kept inside the same repo so the API can spawn prediction subprocesses), and a second copy of the Expo frontend targeted at native mobile. It trades the operational simplicity of a static file for richer request/response semantics.

10.1 Architecture

The Elysia backend exposes a small REST API. The frontend calls `POST /api/forecast` with a body `{model, days, cycles, startDate}`; the backend spawns a Python subprocess `prediction/forecast.py --model balanced_rf --days 30 --cycles 1`, which writes a JSON file to `experiments/forecasts/`; the backend reads the file and returns it. Other endpoints include `GET /api/health` (liveness probe), `GET /api/results/leaderboard` (model comparison), and `POST /api/predict` (single-row prediction). The backend is deployed on Render via a Docker image that installs Bun and Python 3, sets up a virtualenv, and downloads pre-trained model pickles from Hugging Face at build time. The frontend is the same Expo app, repointed via `EXPO_PUBLIC_API_URL`.

10.2 Five-model benchmark

Table 10. Five-model leaderboard from the Elysia backend (`experiments/results/Leaderboard.json`). Metrics are on a single held-out random 70/15/15 split; the Balanced Random Forest dominates because it natively resamples the minority class. High accuracy coexists with very low precision for some models — the test split is heavily imbalanced.

Rank	Model	Accuracy	Precision	Recall	F1	ROC-AUC
1	Balanced Random Forest	1.0000	0.9998	1.0000	0.9999	1.0000
2	LightGBM	0.9981	0.4815	1.0000	0.6500	0.9990
3	KAN	0.9982	0.0000	0.0000	0.0000	—
4	MLP Neural Network	0.9982	0.0000	0.0000	0.0000	—
5	XGBoost	0.9982	0.0000	0.0000	0.0000	0.8777

The leaderboard reveals the same calibration lesson (Section 5.1) in a different form: KAN, MLP, and XGBoost all achieve ≥ 0.998 accuracy yet have zero precision and recall on the minority (heatwave) class — they have collapsed to predicting the majority class. Only Balanced Random Forest and LightGBM carry usable minority-class signal. We do not interpret these as a fair skill comparison with the contract-first pipeline: the Elysia split is a random 70/15/15 rather than a leak-free rolling-origin split, so its numbers are optimistic in the same way random k-fold CV is on autocorrelated meteorological data.

10.3 Configuration and reproducibility

All hyperparameters live in a single `config.yaml`: `balanced_rf` uses 200 trees with `max_depth=15`; `xgboost`/`lightgbm` use 300 estimators, `max_depth=6`, learning rate 0.1, subsampling 0.8; `mlp` has hidden layers [256, 128, 64] with dropout 0.3; `kan` has hidden layers [64, 32], grid size 5, spline order 3. The config pins the split (`train=0.7`, `val=0.15`, `test=0.15`, `random_state=42`, `stratify=true`) and the heat-index labelling thresholds (caution 27°C, extreme caution 32°C, danger 41°C, extreme danger 54°C). This heat-index label differs from the percentile label in Section 2.2; the two deployments solve related but not identical problems.

10.4 Trade-offs vs. the contract-first pipeline

- **Operational cost.** The contract-first pipeline has no always-on server (GitHub Pages free tier). Elysia requires a continuously running Render instance (free-tier hibernation introduces 30+ second cold starts).
- **Latency.** The contract-first frontend reads a cached JSON file in one round-trip; the Elysia frontend blocks on a Python subprocess spawn plus inference (1–5 s for `balanced_rf` on 30 days).
- **Schema enforcement.** The contract-first pipeline has a machine-checked schema and a five-category readiness gate; the Elysia API delegates validation to Elysia's runtime type system and ad-hoc JSON-shape checks.
- **Skill evaluation.** The contract-first pipeline reports a true OOS BSS of +0.178; the Elysia leaderboard reports random-split accuracy on a different label definition — the numbers are not directly comparable.

11 Limitations and Threats to Validity

- **Narrow OOS window.** The OOS backtest covers only 26 origination dates (Jan–Feb 2024) because the upstream MJO RMM feed (BoM) ceased updates on 24 February 2024. Per-lead CIs are too narrow to trust; we report only the overall OOS BSS with its CI. Connecting an alternative MJO source (e.g. NCEP) is the highest-priority operational follow-up.
- **CV optimism.** Research-phase CV BSS values are uniformly optimistic relative to OOS, with the gap largest at leads 5–6. We treat CV as evidence of potential skill and OOS as the gating criterion.
- **Single OOS season.** The window spans one Jan–Feb season only. Multi-season replication (especially a May–June peak) is needed before the operational BSS point estimate can be considered stable.
- **Explainability scope.** We use permutation importance and leave-one-group-out ablation rather than SHAP or LIME. Permutation importance is robust for probabilistic skill but does not provide per-instance attribution.
- **Temporal scope.** Training data cover January–July of each year; the system does not forecast the second half of the calendar year.
- **Modest skill at lead 4.** Released lead 4 has OOS BSS $+0.073$ — positive and significant in aggregate but small in absolute terms. We frame it as a "risk indicator" rather than a "deterministic forecaster"¹⁰.

12 Conclusion

We presented HeatMAP, a deployed end-to-end system for sub-seasonal heatwave forecasting across the 77 provinces of Thailand. Four engineering choices make the system trustworthy enough to release. A rolling-origin CV protocol with a **49-day embargo gap** prevents the temporal leakage that random k-fold splits introduce on autocorrelated meteorological data. A calibration-first evaluation exposes the failure mode in which high AUC coexists with negative BSS, and shows that Platt calibration on a time-separated validation split restores usable skill. A true out-of-sample operational backtest gates which lead times are released: only leads 2–4 pass both $BSS > 0$ and $AUC > 0.5$, and lead 4 is explicitly flagged as weakening. A versioned JSON contract with a machine-enforced validator and a five-category production-readiness gate guarantees that no stale, malformed, or unskillful artifact reaches the public frontend.

The released system achieves overall OOS BSS $+0.178$ [95% CI: $+0.065$, $+0.232$] on never-seen January–February 2024 data, with 10,010 forecast pairs across 77 provinces and three released leads. It is deployed as a serverless weekly cron pipeline producing a single JSON file consumed by a static Expo web frontend, with a validation gate that halts publication on any data-quality failure. The system has been operational since June 2026 and is publicly accessible at heat-map-frontend.vercel.app.

Three broader lessons generalise. First, for rare-event probabilistic tasks where downstream decisions depend on the probability value, AUC alone is insufficient; a calibrated skill score such as BSS is the relevant measure. Second, the gap between CV optimism and OOS realism is real and operationally

consequential: a responsible-release policy must be gated on the latter. Third, a contract-first deployment with a hard validator is the cheapest insurance against the most common production failure modes (stale data, schema drift, over-escalation). Future work will (i) integrate an alternative MJO source to widen the OOS window, (ii) extend the OOS evaluation across multiple seasons including the May–June peak, (iii) explore probabilistic multi-class risk tiers, and (iv) reconcile the contract-first pipeline with the alternative Elysia deployment so its five-model benchmark is re-trained under the same leak-free rolling-origin protocol and percentile-based label.

Appendix A Integrity audit detail

The full integrity audit is in `docs/INTEGRITY.md` of the `DeepSeek_Heatwave` repository. Two leaks were investigated before declaring the system production-ready.

R2 — forward-looking `in_hw_today` (fixed). The `flag_heatwaves` function counts runs in both directions (correct for label generation), but the same output was reused as the `in_hw_today` feature and persistence-baseline state, causing the feature to use future information at origination. The fix replaced it with a trailing-only run-length counter. Re-running the CV showed BSS changes ≤ 0.005 at every lead.

R1 — percentile-label leak (documented, kept). The 90th-percentile threshold was originally computed on the full 30-year history (including the test fold). The maximum $|\Delta\text{BSS}|$ between baked and leak-free variants was 0.0178 at lead 3, well within the 95% CI half-width of every lead (minimum 0.060). The leak is statistically undetectable; we keep the frozen-all-history labels because the BSS ratio cancels the bias (the climatology baseline uses the same labels).

References

1. G. W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.
2. J. C. Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, 61–74. MIT Press, 1999.
3. A. Niculescu-Mizil and R. Caruana. Predicting good probabilities with supervised learning. In *Proc. 22nd ICML*, 625–632, 2005.
4. G. Ke et al. LightGBM: A highly efficient gradient boosting decision tree. In *NeurIPS 30*, 3146–3154, 2017.
5. H. Hersbach et al. The ERA5 global reanalysis. *Q. J. R. Meteorol. Soc.*, 146(730):1999–2049, 2020.
6. M. C. Wheeler and H. H. Hendon. An all-season real-time multivariate MJO index. *Monthly Weather Review*, 132(8):1917–1932, 2004.
7. K. E. Trenberth. The definition of El Niño. *Bull. Amer. Meteor. Soc.*, 78(12):2771–2777, 1997.
8. Y. Benjamini and Y. Hochberg. Controlling the false discovery rate. *J. R. Stat. Soc. B*, 57(1):289–300, 1995.
9. D. S. Wilks. *Statistical Methods in the Atmospheric Sciences*, 3rd ed. Academic Press, 2011.
10. F. Vitart, A. W. Robertson, D. L. T. Anderson. Subseasonal to seasonal prediction project. *WMO Bulletin*, 61(2):23–28, 2012.
11. S. E. Perkins and L. V. Alexander. On the measurement of heat waves. *J. Climate*, 26(13):4500–4517, 2013.

12. H. R. Künsch. The jackknife and the bootstrap for general stationary observations. *Ann. Statist.*, 17(3):1217–1241, 1989.
13. H. Chantry et al. Opportunities for the use of machine learning in subseasonal to seasonal forecasting. *Geophys. Res. Lett.*, 48(12):e2021GL093683, 2021.
14. S. M. Lundberg and S.-I. Lee. A unified approach to interpreting model predictions. In *NeurIPS 30*, 4765–4774, 2017.
15. Y.-Y. Hwang et al. Improving subseasonal forecast skill of the MJO using a stochastic multicloud model. *Monthly Weather Review*, 147(7):2603–2619, 2019.
16. B. Casati et al. Forecast verification: current status and future directions. *Meteorol. Appl.*, 15(1):3–18, 2008.
17. C. Zhang. Madden–Julian Oscillation: bridging weather and climate. *Bull. Amer. Meteor. Soc.*, 94(12):1849–1870, 2013.
18. A. K. Betts et al. The land surface–atmosphere interaction: a review. *J. Geophys. Res. Atmos.*, 101(D3):7209–7225, 1996.
19. S. I. Seneviratne et al. Investigating soil moisture–climate interactions in a changing climate. *Earth-Sci. Rev.*, 99(3–4):125–161, 2010.
20. A. W. Robertson et al. Quantifying predictability of the low-frequency atmosphere for S2S prediction. *Climate Dynamics*, 45(3–4):939–948, 2015.
21. J. Peng et al. A review of spatial downscaling of satellite remotely sensed soil moisture. *Rev. Geophys.*, 55(2):341–366, 2017.
22. Y. Guo et al. Heat wave and mortality: a multicountry, multicomunity study. *Environ. Int.*, 99:233–242, 2017.
23. Y. H. Lim et al. Effects of heat waves on daily mortality in two cities of Thailand. *Int. J. Environ. Res. Public Health*, 15(10):2248, 2018.
24. T. R. Scully, P. Smet, K. Chen. Low-cost deployment of machine-learning models using static-file serving. *Softw. Pract. Exper.*, 52(10):2101–2123, 2022.
25. L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
26. T. Chen and C. Guestrin. XGBoost: a scalable tree boosting system. In *Proc. 22nd ACM SIGKDD*, 785–794, 2016.
27. D. P. Kingma and J. Ba. Adam: a method for stochastic optimization. *arXiv:1412.6980*, 2014.
28. Z. Liu et al. KAN: Kolmogorov–Arnold networks. *arXiv:2404.19756*, 2024.